

Midterm System Presentation & Security Review

The state of the system after D1, D2 and D3

Team 05 — CypherPunks

· Arroyo Ramírez Carlos Alberto
· Escudero Bohórquez Julio
· Martínez Miranda Juan Carlos
· Pérez Avin Paola Celina de Jesús
· Salgado Miranda Jorge

Criptografía · Group 04

Two people exchange a file through channels and storage that neither of them controls.

01

Confidentiality



Only the intended recipients can read the file, even if the package is exposed.

02

Integrity



Any change to content, metadata or recipient list is detected and nothing is released.

03

Controlled sharing



A file is encrypted once and opened by several named recipients, each with their own key.

Built so far: **D2** single-owner AEAD encryption · **D3** hybrid multi-recipient sharing · Python CLI

B — ARCHITECTURE

Trusted endpoints, untrusted middle

Plaintext and clear keys exist only inside each Vault. The Secure Package crosses the untrusted zone and must protect itself.

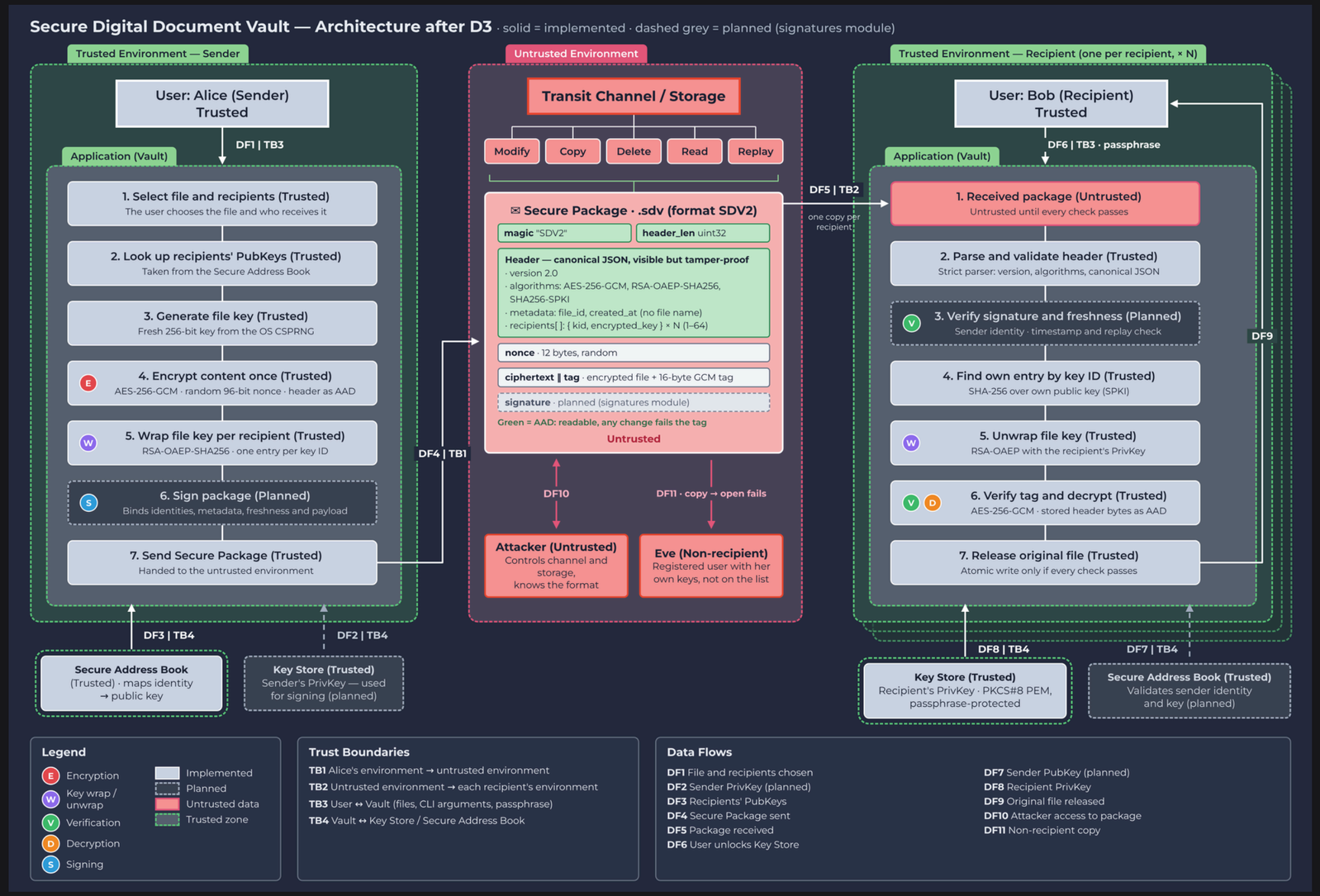
TB1 Sender ⟶ untrusted zone

TB2 Untrusted ⟶ each recipient

TB3 User ↔ Vault

TB4 Vault ↔ Key Store / Address Book

Dashed grey boxes are planned for the signatures module.



Encrypt  → share  → decrypt 

TRUSTED · SENDER VAULT

Fresh 256-bit file key
AES-256-GCM, once
Wrap key per recipient

→

UNTRUSTED · .SDV (SDV2)

Header (AAD): algorithms, file_id, created_at, recipients
Nonce · ciphertext || tag

→

TRUSTED · RECIPIENT VAULT

Find own key ID
Unwrap with private key
Verify tag, then decrypt

Component	Today in the repository
Vault	Python CLI: encrypt / decrypt (D2) · keygen / share / open / recipients (D3)
Key Store	RSA-3072 private key as PKCS#8 PEM, always encrypted with a passphrase
Address Book	Recipient public keys as .pub.pem files, identified by a SHA-256 key ID

Attackers described by what they can do

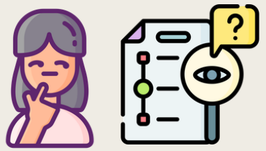
A1



Network / storage attacker

Controls the untrusted zone and knows the format. Can **read, copy, modify, replace, replay or delete** packages and craft malformed ones.

A2



Non-recipient user (Eve)

A legitimate user with her own key pair who obtains a file not shared with her and tries to **open it or add herself** as a recipient.

Asset we protect	Property that must hold
File contents	Confidentiality and integrity
Protected file key	Recoverable only with a listed recipient’s private key
Metadata and recipient list	No undetected modification
Private keys	Never leave the Key Store in usable form

What we assume, and what is out of scope

We assume

- Endpoints are not compromised while the Vault runs
- The crypto library implements AES-GCM and RSA-OAEP correctly
- The OS random generator is secure
- Public keys come from a trusted source (Address Book)
- Every received package is hostile until checked



Out of scope

- Availability: an attacker can delete a package
- Malware on Alice’s or Bob’s device
- What Bob does with the plaintext
- Traffic analysis
- Key revocation, rotation and enrollment



If the Address Book hands us a wrong public key, encryption still succeeds, but for the wrong person.

AES-256-GCM

Why this algorithm over ChaCha20-Poly1305

- Both candidates meet every D2 requirement. The choice came down to standardization and the hardware we run on.
- NIST-standardized, and our machines have AES-NI, so it is the faster and safer implementation for us. Nonce reuse is equally fatal in both, and we avoid it with a fresh key per file.



	AES-256-GCM	ChaCha20-Poly1305
Standard	NIST SP 800-38D	IETF RFC 8439
Key / nonce / tag	256-bit / 96-bit / 128-bit	256-bit / 96-bit / 128-bit
Speed with AES-NI	Fastest option	Slower than AES-GCM
Speed without AES hardware	Slow	Fast and constant-time in pure software
Nonce reuse	Catastrophic: XOR leak + tag forgery	Catastrophic: XOR leak + tag forgery
Max size per message	≈ 64 GiB	≈ 256 GiB
In our library	AESGCM	`ChaCha20Poly1305`

`256-bit key · fresh key per file from the OS CSPRNG · 96-bit random nonce · 16-byte tag`

Nonce strategy and metadata as AAD

magic	length	header (canonical JSON)	nonce	ciphertext || tag
"SDV2"	uint32 BE	algorithm, version, timestamp…	12 bytes	tag = 16 bytes

AAD: read in clear, but any change fails the tag

Nonce



96 bits from **os.urandom(12)**, generated inside **encrypt()**: the caller cannot fix or reuse it. One key encrypts one file, so a repeat under the same key cannot happen.

Metadata + AAD



The whole header is the AAD, taken from the raw stored bytes. JSON is parsed only after the tag verifies. **encrypt()** refuses empty AAD.

Hybrid workflow: encrypt once, wrap per recipient

FILE KEY

256 bits

Fresh, from the OS CSPRNG.
Never written to disk.

`AES-256-GCM`	File ⟶ one ciphertext ‖ tag, header as AAD
`RSA-OAEP · Alice`	File key ⟶ 384-byte wrapped key for Alice
`RSA-OAEP · Bob`	File key ⟶ 384-byte wrapped key for Bob

Why hybrid: RSA solves key distribution but encrypts only a few hundred bytes and is slow, AES handles the data.

Parameters: RSA-3072, OAEP with SHA-256 and MGF1, label “SDV2/file-key”. Only 32-byte keys may be wrapped.

Recipients and key identification

```
{"algorithms": {
  "aead": "AES-256-GCM",
  "key_wrap": "RSA-OAEP-SHA256",
  "key_id": "SHA256-SPKI"},
"metadata": {"created_at", "file_id"},
"recipients": [
  {"kid": "<64 hex>",
   "encrypted_key": "<b64>"}, …],
"version": "2.0"}
```

How keys are identified

Key ID = SHA-256 of the DER public key (SPKI). Deterministic, reveals nothing about the private key; the recipient looks for exactly one match.

How recipients are handled

Sorted by key ID, 1–64 entries, duplicates rejected. The whole list is part of the AAD.

Strict parser

Unknown fields, versions or algorithms and non-canonical JSON are rejected: no downgrade.

Every failure looks the same, and nothing is written

Attacker action	Result
Opens a shared file without being a recipient	“authentication failed”, no output
Adds own entry, removes or swaps recipients	Tag fails for every recipient
Flips bits in ciphertext, tag, nonce or header	“authentication failed”, no output
Changes an algorithm identifier (downgrade)	Rejected by the parser
Truncates or corrupts the structure	Rejected before any cryptography
Uses a wrong private key	OAEP unwrap fails, same generic error

> Output goes through a temporary file and an atomic rename, so a failed check never leaves partial plaintext. One generic error avoids giving an oracle about which check failed.

Four scenarios, one terminal

1	**Share with Alice and Bob**	`vault share reporte.pdf -r alice.pub.pem -r bob.pub.pem`	**succeeds**
2	**Bob decrypts**	`vault open shared.sdv -i bob.key.pem`	**identical PDF**
3	**Eve is not a recipient**	`vault open shared.sdv -i eve.key.pem`	**fails, no file**
4	**One byte modified**	`vault open tampered.sdv -i bob.key.pem`	**fails, no file**

Single-owner D2 flow as backup: **vault encrypt** / **vault decrypt -k**

What holds today and what D1 still asks for

D1 requirement	Status	What's next
SR-01 Confidentiality	Done (AES-256-GCM + RSA-OAEP)	
SR-02 / SR-03 Integrity, tampering	Done (Header and recipients as AAD)	
SR-06 Safe failure, ordered checks	Done (Atomic write, generic error)	
SR-07 Metadata minimization	Done in SDV2 (SDV1 shows file name)	
SR-04 Sender authenticity	Pending	Digital signatures
SR-05 Freshness (replay)	Pending	Validate signed timestamp / file_id
SR-10 Authentic public keys	Pending	Certificates / trusted Address Book